Report Esercizio W13D1 - File Upload su DVWA

Oct 17, 2026 - @Donato

Studente	Squicciarino Donato
Corso	Epicode - Cybersecurity & Ethical Hacker
Esercizio	W13D1 - File Upload su DVWA
Scopo	Sfruttare la vulnerabilita di File Upload della DVWA per caricare ed eseguire una shell PHP, intercettando ogni richiesta con BurpSuite

1. Obiettivo dell'esercizio

L'esercizio W13D1 del corso Epicode "Cybersecurity & Ethical Hacker" ha come scopo l'analisi pratica di una vulnerabilita di tipo **Unrestricted File Upload**, una delle categorie piu diffuse nella sicurezza delle applicazioni web (OWASP Top 10).

L'obiettivo didattico e duplice:

1. Comprendere come un meccanismo di caricamento file privo di controlli adeguati sul tipo e sul contenuto del file possa essere sfruttato per ottenere l'esecuzione di codice da remoto (Remote Code Execution) sul server che ospita l'applicazione.
2. Familiarizzare con l'uso di **BurpSuite** come proxy di intercettazione, per osservare ed analizzare nel dettaglio ogni richiesta HTTP scambiata tra client e server durante l'exploit.

L'ambiente di laboratorio utilizzato e composto da una macchina **Kali Linux** (attaccante) e una macchina **Metasploitable** (vittima), sulla quale e installata l'applicazione volutamente vulnerabile **DVWA (Damn Vulnerable Web Application)**, con livello di sicurezza impostato su *low*.

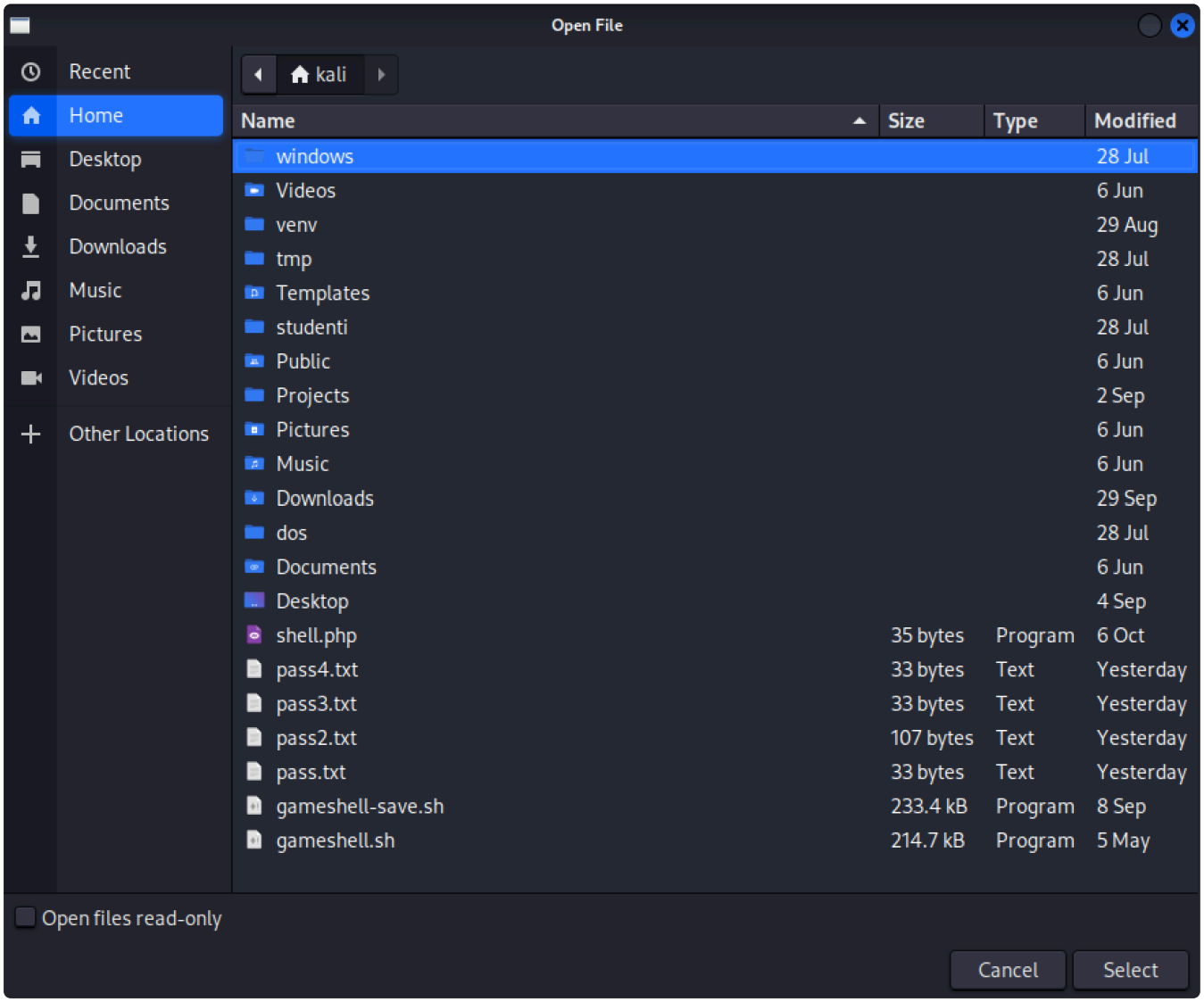
2. Preparazione del file da caricare

Il primo passaggio consiste nel creare, sulla macchina Kali Linux, il file che verra poi caricato sul server vulnerabile. Il file e stato creato direttamente da terminale con l'editor di testo **nano** (comando `nano shell.php`), digitando al suo interno una singola riga di

codice PHP minimale che, una volta eseguita dal server, legge il parametro `cmd` passato in una richiesta ed esegue il comando di sistema corrispondente, restituendone l'output.

L'uso di nano, al posto di un editor grafico, rispecchia l'approccio tipico di un ambiente di penetration testing: l'intera catena di lavoro (creazione del payload, upload, interazione) avviene da riga di comando, in modo rapido e ripetibile.

Una volta salvato il file, questo e stato verificato nella cartella Home dell'utente *kali*, insieme agli altri file di supporto dell'esercizio, pronto per essere selezionato dal modulo di upload della DVWA.



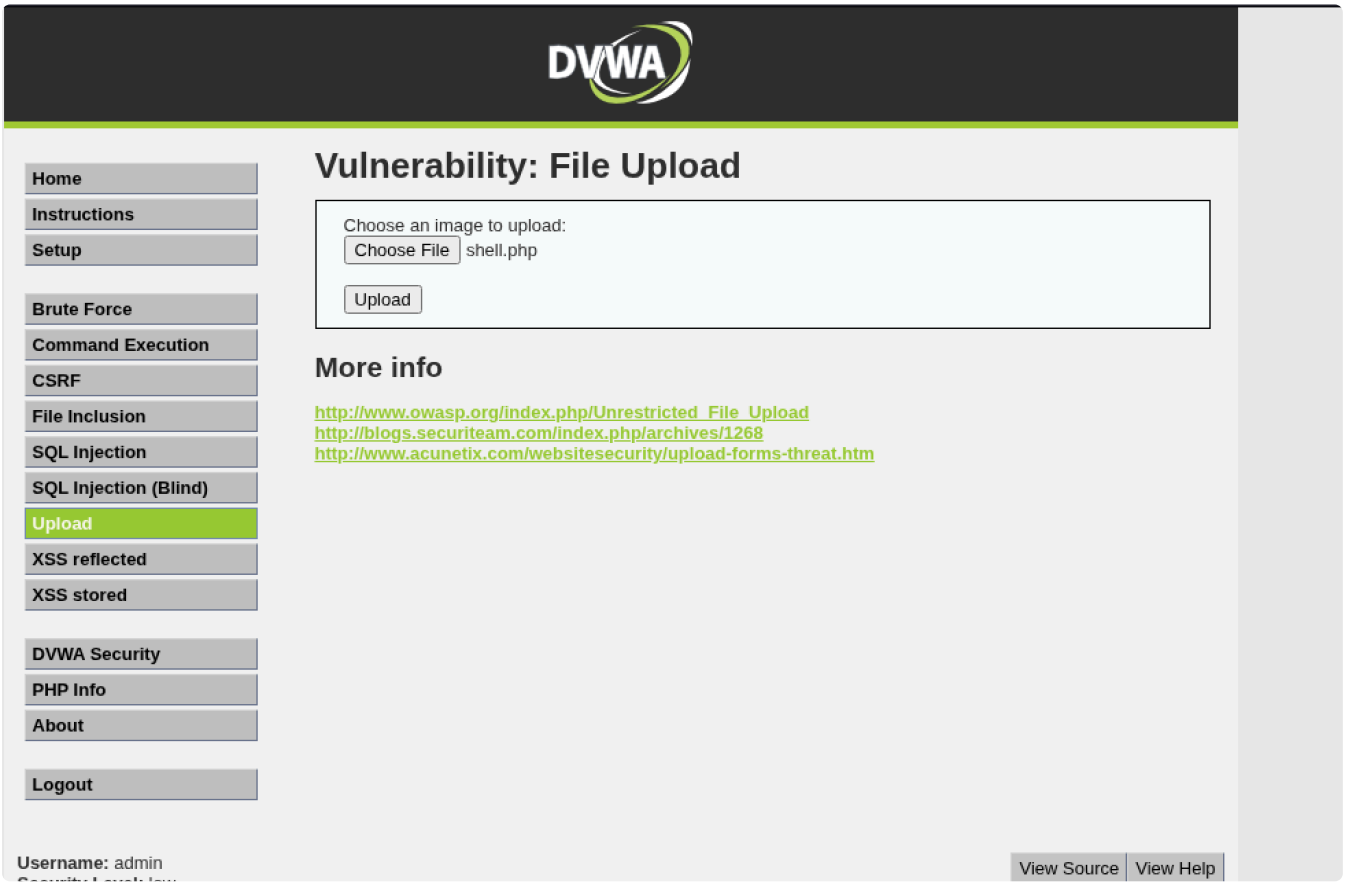
3. Caricamento del file tramite la vulnerabilita File Upload

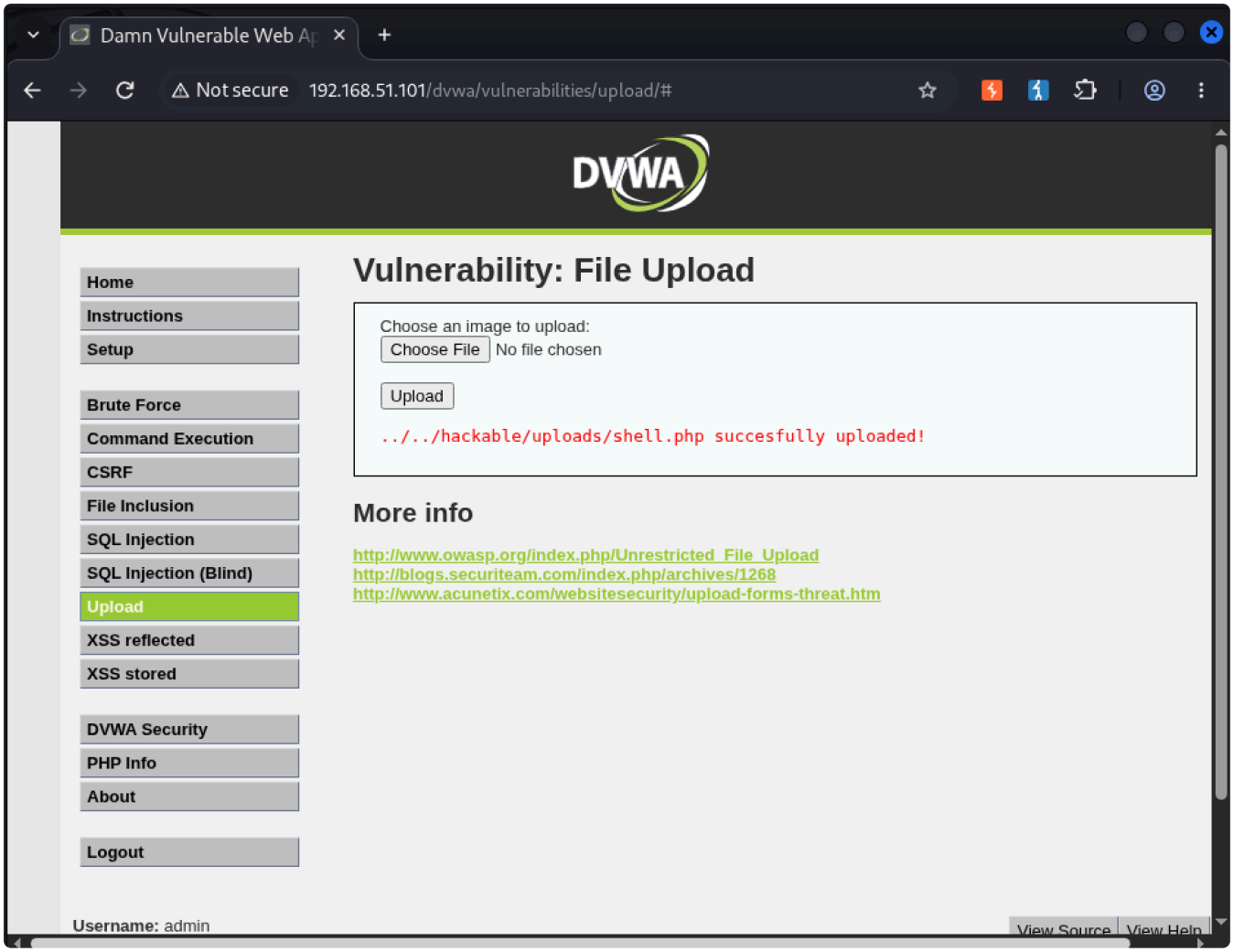
Con il security level della DVWA impostato su *low*, ci si e posizionati sulla sezione **Upload**. La pagina presenta un semplice modulo "Choose an image to upload", senza alcun controllo lato server sull'estensione o sul tipo MIME del file selezionato: questa e la radice della vulnerabilita, poiche il modulo accetta qualsiasi file, incluso un file eseguibile come `.php`.

Dopo aver selezionato il file tramite il pulsante *Choose File* e confermato l'invio con *Upload*, la DVWA ha restituito il messaggio di conferma:

```
../../hackable/uploads/shell.php succesfully uploaded!
```

Questo messaggio indica che il file e stato salvato sul server all'interno della cartella `hackable/uploads/`, raggiungibile direttamente via browser. Le due schermate seguenti mostrano, nell'ordine, il modulo prima dell'invio (con il file selezionato) e la risposta del server dopo il caricamento andato a buon fine.





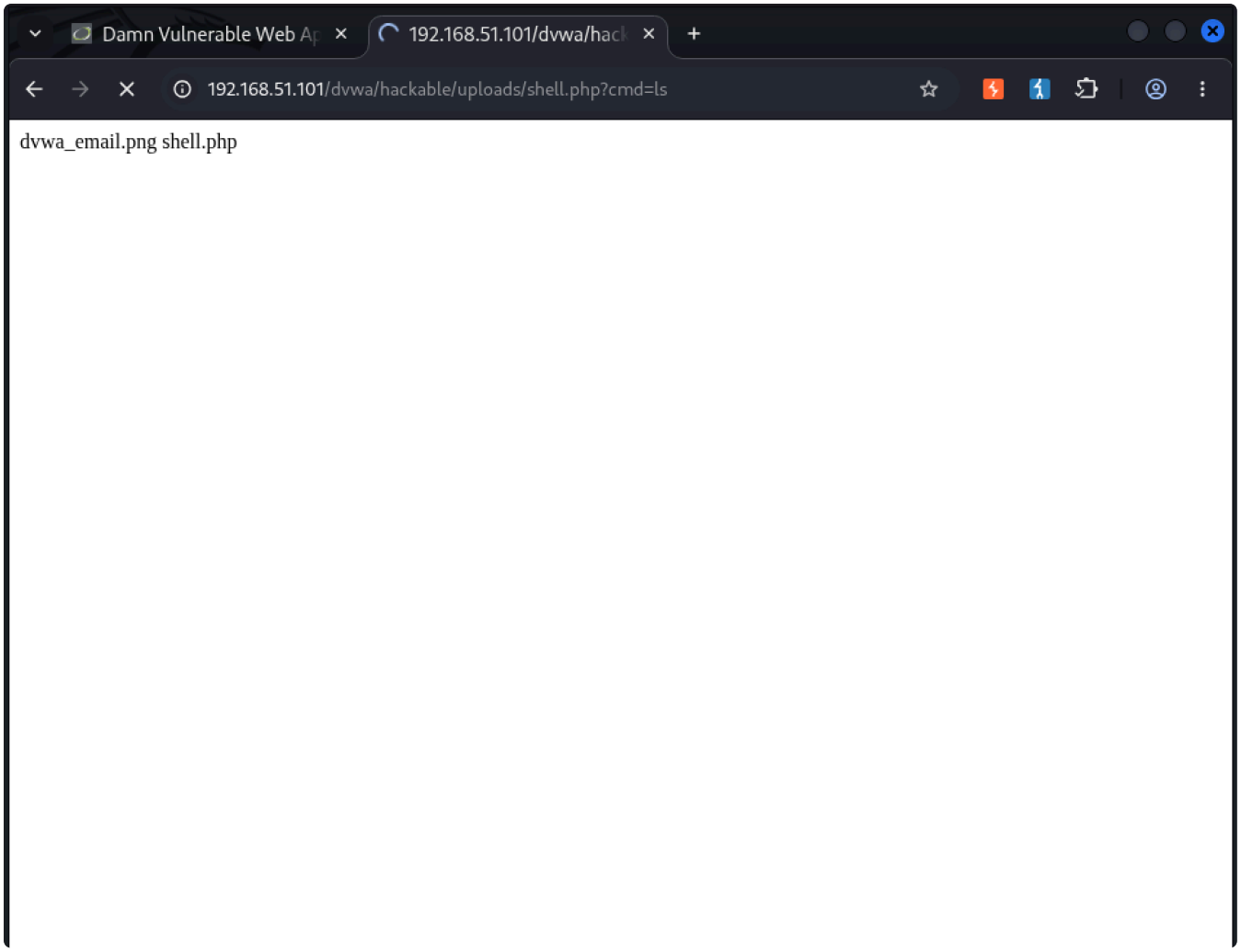
4. Verifica del caricamento ed esecuzione di comandi da remoto

Una volta confermato il caricamento, il file e stato raggiunto direttamente dal browser all'indirizzo `http://192.168.51.101/dvwa/hackable/uploads/shell.php`, aggiungendo il parametro `?cmd=ls` alla query string.

Il server ha eseguito il comando `ls` all'interno della cartella `uploads/` e ne ha restituito l'output direttamente nella pagina:

```
dvwa_email.png shell.php
```

Questo risultato dimostra concretamente l'impatto della vulnerabilita: non si tratta piu soltanto di aver caricato un file sul server, ma di aver ottenuto la capacita di eseguire comandi di sistema arbitrari da remoto, semplicemente modificando il parametro `cmd` nell'URL (ad esempio con `whoami`, `id`, `pwd` o comandi piu invasivi). E il passaggio che trasforma una semplice falla di upload in una compromissione completa del server (Remote Code Execution).



5. Intercettazione e analisi con BurpSuite

Come richiesto dalla traccia, l'intera interazione con la DVWA e stata monitorata tenendo attiva una sessione di intercettazione su **BurpSuite** (Community Edition), con il proxy configurato sul browser e l'intercettazione in ascolto nel pannello *Proxy > Intercept*.

La schermata seguente mostra la richiesta GET intercettata nel momento dell'esecuzione del comando sulla shell caricata:

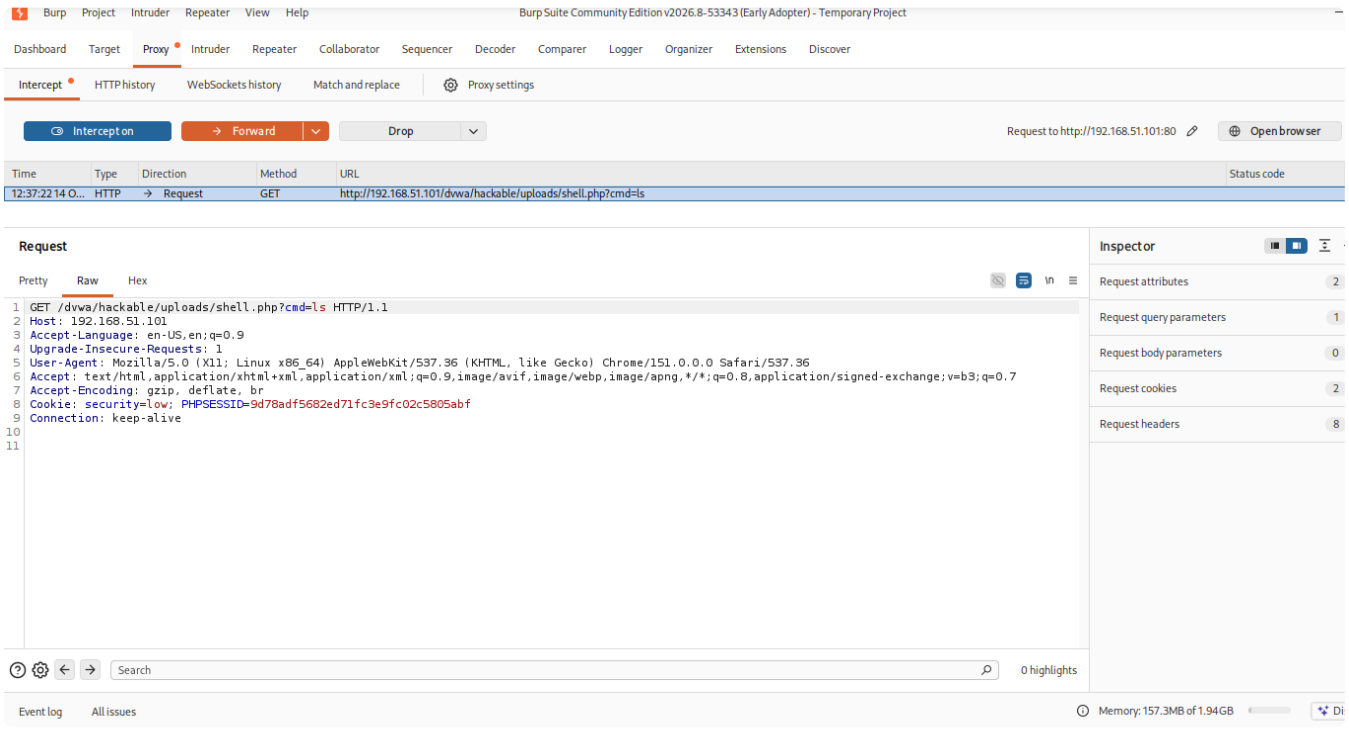
```
GET /dvwa/hackable/uploads/shell.php?cmd=ls HTTP/1.1
```

Dall'analisi della richiesta grezza (tab *Raw*) e possibile osservare alcuni dettagli rilevanti:

- Il parametro `cmd` viene trasmesso in chiaro all'interno della **query string** dell'URL (metodo GET), non nel corpo della richiesta: questo significa che ogni comando eseguito resta visibile nella cronologia del browser, nei log del server e in qualunque strumento di intercettazione, rendendo l'attacco facilmente ricostruibile a posteriori.
- L'header `Cookie` riporta `security=low`, confermando il livello di sicurezza impostato nella DVWA, insieme al `PHPSESSID` che identifica la sessione autenticata dell'utente *admin*.

- Non e presente alcun meccanismo di protezione (token CSRF, validazione del referer, rate limiting) che impedisca l'invio ripetuto di richieste con comandi diversi.

L'uso di BurpSuite in questa fase ha permesso di passare dalla semplice osservazione del risultato nel browser a una comprensione precisa del protocollo HTTP sottostante, elemento fondamentale nel lavoro di un penetration tester.



6. Conclusioni

L'esercizio ha confermato in pratica quanto una vulnerabilita di File Upload priva di controlli sia critica: la mancata verifica dell'estensione e del contenuto del file caricato ha permesso di trasformare un semplice modulo "upload immagine" in un vettore di esecuzione di comandi arbitrari sul server, con impatto equivalente a una compromissione completa della macchina.

L'analisi con BurpSuite ha inoltre evidenziato come l'intero attacco si fondi su richieste HTTP del tutto ordinarie (una GET con un parametro), il che lo rende difficile da distinguere da traffico legittimo in assenza di controlli specifici lato server o di sistemi di rilevamento (WAF, IDS).

Dal punto di vista della remediation, le contromisure principali per questo tipo di vulnerabilita includono: whitelist rigorosa delle estensioni consentite, verifica del tipo MIME reale del file (non solo dell'estensione dichiarata), rinomina casuale dei file caricati, salvataggio al di fuori della webroot o in una cartella senza permessi di esecuzione, e scansione antimalware dei contenuti caricati.

L'esercizio ha permesso di consolidare sia l'aspetto offensivo (sfruttamento pratico della vulnerabilita) sia quello di analisi del traffico di rete, entrambi elementi chiave nel percorso di formazione come Ethical Hacker.